



**KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN MẠNG VÀ CÁC HỆ THỐNG THÔNG TIN**

CHƯƠNG 7

BỘ NHỚ ẢO

Nội dung chính - Chương 7



Mục lục

- 7.1. Đặt vấn đề
- 7.2. Phân trang theo yêu cầu
- 7.3. Thay thế trang
- 7.4. Cấp phát khung
- 7.5. Phân đoạn theo yêu cầu

Từ khóa

Virtual memory

Demand paging

Page fault

Page replacement

FIFO

OPT

LRU

Clock

Frame allocation

Thrashing

Working set

Demand segmentation

Trọng tâm kiến thức



Hiểu ý tưởng bộ nhớ ảo

Chương trình có thể chạy dù chỉ một phần đang nằm trong RAM.

Thực hiện thuật toán

Mô phỏng lỗi trang và các thuật toán FIFO, OPT, LRU, Clock.

Nhận biết quá tải

Giải thích cấp phát khung, tập làm việc và hiện tượng thrashing.

Câu hỏi định hướng: Làm thế nào chạy chương trình lớn hơn RAM mà hệ thống vẫn phản hồi tốt?

Tình huống mở đầu: Chương trình lớn hơn RAM



Chương trình

Dung lượng 2 GB
Có nhiều chức năng và dữ liệu

RAM còn trống

Chỉ còn 600 MB
Không đủ chứa toàn bộ chương trình

Câu hỏi

Có nhất thiết phải nạp hết
2 GB trước khi chạy?

Ý tưởng

Chỉ nạp phần đang dùng

Khi cần

Nạp thêm trang mới

Khi RAM đầy

Thay một trang ít cần

Đó là ý tưởng cốt lõi của bộ nhớ ảo.



7.1. Đặt vấn đề

Bộ nhớ ảo giúp tách “dung lượng chương trình nhìn thấy” khỏi lượng RAM đang có thật.

Vì sao cần bộ nhớ ảo?



Chương trình ngày càng lớn

Không phải mọi phần của chương trình đều được dùng cùng lúc.

Nhiều tiến trình cùng chạy

RAM phải chia cho hệ điều hành và nhiều ứng dụng.

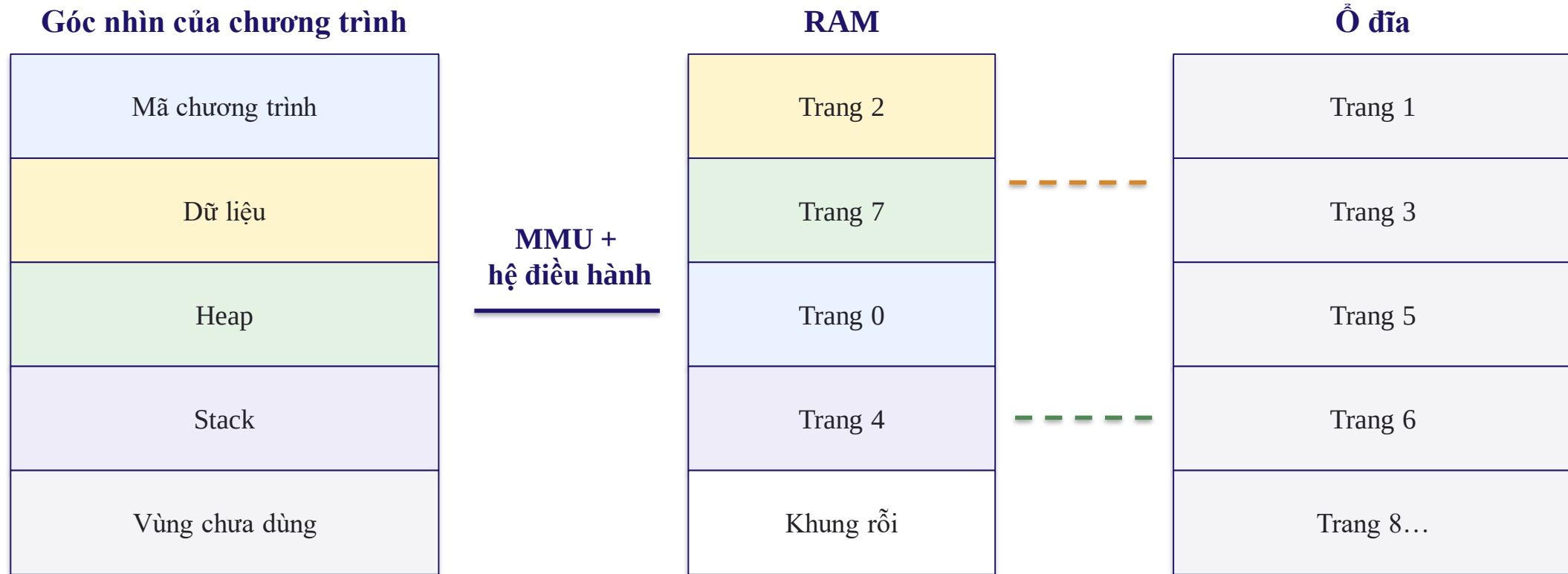
RAM có giới hạn

Không thể luôn lắp thêm RAM để giải quyết mọi trường hợp.

Bộ nhớ ảo cho phép:

- Mỗi tiến trình nhìn thấy một không gian nhớ lớn và riêng.
- Chỉ những phần cần thiết mới được giữ trong RAM.
- Phần còn lại nằm trên ổ đĩa và được nạp khi cần.

Chương trình nhìn thấy một vùng nhớ lớn



Chương trình không cần biết trang nào đang ở RAM và trang nào đang ở ổ đĩa.

Ý tưởng cốt lõi của bộ nhớ ảo



1. Chia nhỏ

Chương trình được chia thành các trang hoặc đoạn.

2. Nạp một phần

Chỉ phần đang cần được đưa vào RAM.

3. Nạp khi thiếu

Truy cập phần chưa có sẽ tạo lỗi trang.

4. Thay khi đầy

Chọn một phần cũ để nhường khung.

Lợi ích

Chạy được chương trình lớn hơn RAM và tăng số tiến trình cùng hoạt động.

Đánh đổi

Nếu lỗi trang xảy ra quá nhiều, ổ đĩa trở thành nút thắt và máy rất chậm.



Tính cục bộ: Chương trình thường chỉ dùng một vùng nhỏ

Cục bộ theo thời gian

Một lệnh hoặc dữ liệu vừa dùng thường sẽ được dùng lại sớm.

Cục bộ theo không gian

Địa chỉ gần phần vừa dùng thường cũng sắp được truy cập.

Ví dụ

Vòng lặp, mảng, hàm đang chạy và dữ liệu của hàm.

Chuỗi truy cập minh họa

3

4

5

4

3

4

12

13

12

13

14

13

Nhờ tính cục bộ, chỉ cần giữ “vùng đang hoạt động” trong RAM là chương trình vẫn chạy tốt.

Không cần mọi trang cùng nằm trong RAM



Không gian trang của tiến trình

Trang 0
Trang 1
Trang 2
Trang 3
Trang 4
Trang 5
Trang 6
Trang 7

Đang ở RAM

Trang 0

Trang 2

Trang 4

Trang 6

Chưa ở RAM

Trang 1

Trang 3

Trang 5

Trang 7

Khi CPU truy cập trang 3:

1. Phần cứng thấy trang chưa có trong RAM.
2. Hệ điều hành nạp trang 3 từ ổ đĩa.
3. Cập nhật bảng trang và chạy lại lệnh.



Bảng trang cần biết trang có đang ở RAM hay không

Trang	Khung	Có trong RAM?
0	5	Có
1	—	Chưa
2	1	Có
3	—	Chưa
4	7	Có

Bit hiện diện (present bit)

1: trang đang ở RAM.
0: trang chưa ở RAM.

Khi present = 0

Truy cập trang sẽ tạo lỗi trang để hệ điều hành xử lý.

Bảng trang không chỉ lưu số khung, mà còn lưu trạng thái và quyền truy cập.

Bộ nhớ ảo mang lại lợi ích gì?



Chạy chương trình lớn

Không cần nạp toàn bộ chương trình vào RAM.

Chạy nhiều tiến trình

Mỗi tiến trình chỉ giữ phần đang dùng.

Bảo vệ tốt hơn

Mỗi tiến trình có không gian địa chỉ riêng.

Chia sẻ linh hoạt

Có thể dùng chung mã thư viện hoặc vùng dữ liệu.

Hiệu quả chỉ tốt khi số lần phải đọc trang từ ổ đĩa không quá nhiều.

RAM

Nhanh, nhưng dung lượng hạn chế.

Ổ đĩa

Lớn hơn, nhưng chậm hơn RAM rất nhiều.

Bộ nhớ ảo không phải “RAM miễn phí”



Bảng trang lớn hơn

Cần thêm bộ nhớ để quản lý các trang của mỗi tiến trình.

Lỗi trang rất chậm

Nạp trang từ ổ đĩa chậm hơn truy cập RAM hàng nghìn lần.

Có thể bị thrashing

Nếu thay trang liên tục, CPU phải chờ I/O thay vì chạy chương trình.

Muốn hệ thống nhanh, cần phối hợp tốt ba quyết định:

- Trang nào nên nạp?
- Khi RAM đầy, nên thay trang nào?
- Mỗi tiến trình nên được bao nhiêu khung?



Ví dụ: Chương trình 12 trang, RAM chỉ giữ 4 trang

12 trang của chương trình

P0
P1
P2
P3
P4
P5
P6
P7
P8
P9
P10
P11

4 khung trong RAM

P0
P1
P4
P7

Phần còn lại trên ổ đĩa

P2
P3
P5
P6
P8
P9
P10
P11

Khi chương trình chuyển sang dùng P2, hệ điều hành nạp P2 và có thể phải thay một trang đang ở RAM.



7.2. Phân trang theo yêu cầu

Trang chỉ được nạp vào RAM khi CPU thực sự truy cập đến trang đó.

Phân trang theo yêu cầu là gì?



Ban đầu

Chỉ nạp một số trang cần để tiến trình bắt đầu.

Khi truy cập

Trang đã có trong RAM thì CPU dùng ngay.

Khi thiếu trang

Tạo lỗi trang và nạp trang cần từ ổ đĩa.

CPU

Tạo địa chỉ

Bảng trang

Kiểm tra present

RAM

Có trang → truy cập

Hệ điều hành

Chưa có → nạp trang

Phân trang hoàn toàn theo yêu cầu



Ý tưởng

Khi tiến trình bắt đầu, có thể chưa có trang nào của nó trong RAM.

Lần đầu chạy

Lệnh đầu tiên gây lỗi trang;
hệ điều hành nạp trang chứa lệnh.

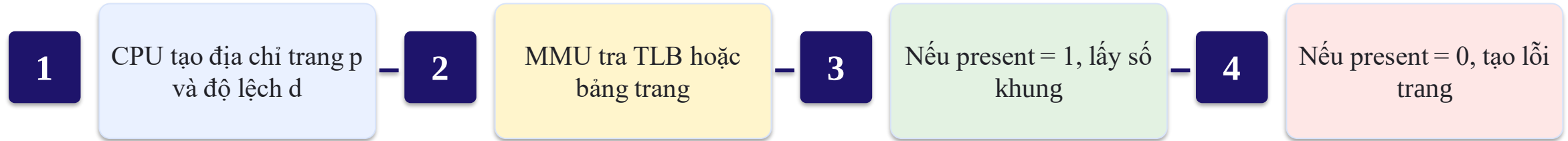
Các lần sau

Mỗi trang mới chỉ được nạp khi chương trình chạm đến.

Cách này tiết kiệm RAM nhất, nhưng giai đoạn đầu có thể xuất hiện nhiều lỗi trang liên tiếp.

Trong thực tế, hệ điều hành thường nạp trước một số trang có khả năng được dùng sớm.

Mỗi lần CPU truy cập bộ nhớ diễn ra như thế nào?



Khi có lỗi trang:

- Hệ điều hành tạm dừng tiến trình.
- Nạp trang từ ổ đĩa vào một khung.
- Cập nhật bảng trang.
- Chạy lại đúng lệnh vừa bị dừng.

Lỗi trang không phải lỗi của chương trình



Lỗi trang (page fault)

Trang hợp lệ của tiến trình nhưng hiện chưa nằm trong RAM.

Truy cập sai

Địa chỉ không thuộc không gian của tiến trình hoặc vi phạm quyền.

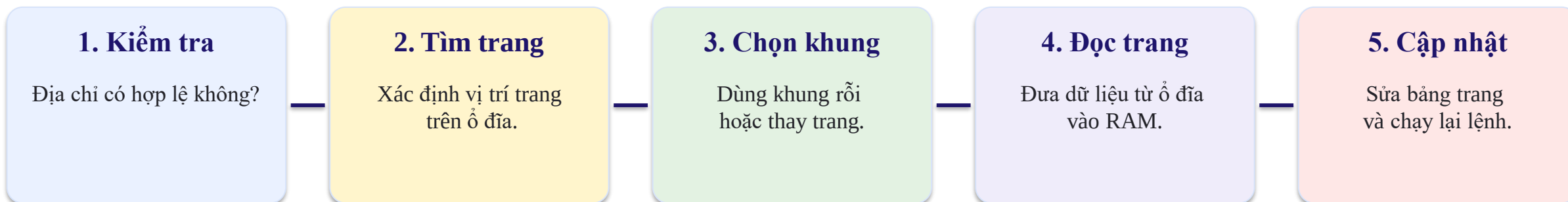
Khác nhau

Lỗi trang có thể xử lý rồi chạy tiếp; truy cập sai thường kết thúc tiến trình.

“Fault” ở đây là một sự kiện để hệ điều hành nạp dữ liệu, không nhất thiết là lỗi lập trình.

Ví dụ: Trang 5 thuộc chương trình nhưng đang ở ổ đĩa → lỗi trang.
Địa chỉ vượt giới hạn tiến trình → lỗi bảo vệ.

Các bước xử lý một lỗi trang

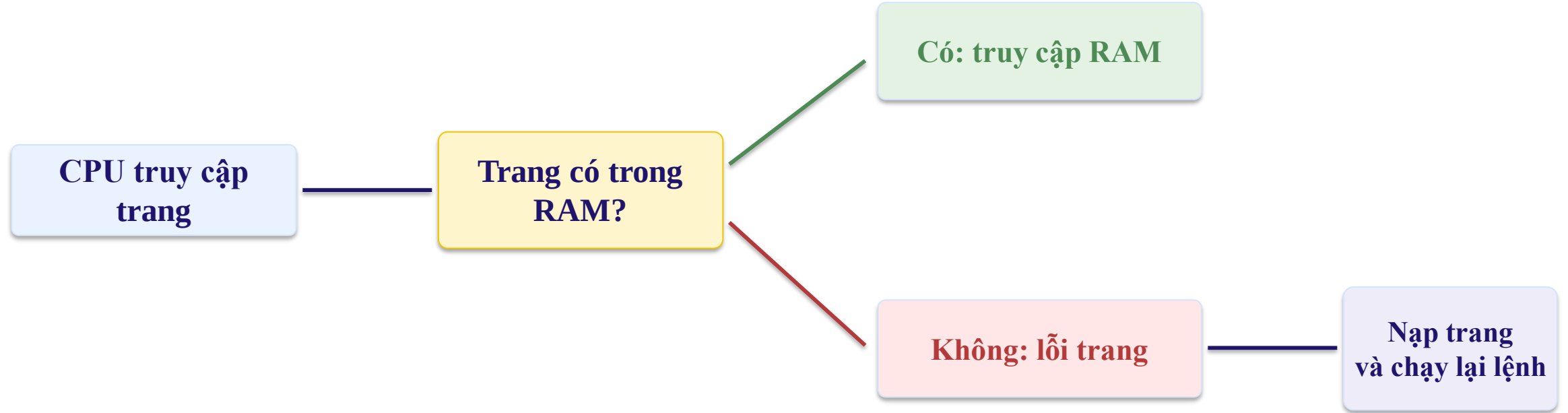


Lệnh gây lỗi trang phải được chạy lại như thể chưa từng thực hiện.

Có khung rồi
Nạp trang trực tiếp vào khung.

Không còn khung rồi
Phải chọn một trang để thay.

Sơ đồ xử lý lỗi trang



Nhánh “Có” thực thi rất nhanh; Nhánh “Không” phải chờ ổ đĩa nên chậm hơn rất nhiều.



Vì sao lỗi trang tốn thời gian?

Dừng tiến trình

CPU chuyển sang hệ điều hành để xử lý sự kiện.

Chờ ổ đĩa

Tìm vị trí, đọc trang và có thể ghi trang cũ.

Khôi phục

Cập nhật bảng trang, TLB và chạy lại lệnh.

Truy cập RAM

≈ vài chục đến vài trăm
ns

Xử lý lỗi trang

≈ vài ms

Chậm hơn rất nhiều

Thời gian truy cập trung bình



$$\text{EAT} \approx (1 - p) \times \text{thời gian RAM} + p \times \text{thời gian xử lý lỗi trang}$$

p

Tỷ lệ truy cập gây lỗi trang.

Thời gian RAM

Rất nhỏ, thường tính bằng ns.

Thời gian lỗi trang

Lớn, thường tính bằng ms.

Chỉ một tỷ lệ lỗi trang rất nhỏ cũng có thể làm thời gian trung bình tăng mạnh.



Ví dụ tính thời gian truy cập trung bình

Dữ liệu

RAM: 100 ns
Lỗi trang: 8 ms

Tỷ lệ lỗi trang

$$p = 1 / 10.000 \\ = 0,0001$$

Kết quả

EAT $\approx$ 900 ns
Chậm gần 9 lần

Cách tính:

$$(1 - 0,0001) \times 100 + 0,0001 \times 8.000.000 \\ \approx 899,99 \text{ ns}$$

Hệ điều hành phải giữ tỷ lệ lỗi trang thật thấp.

Phần cứng và hệ điều hành cần hỗ trợ gì?



Bảng trang

Lưu số khung, bit hiện diện và quyền truy cập.

Vùng lưu trên đĩa

Giữ các trang chưa nằm trong RAM.

Lệnh chạy lại được

Lệnh bị ngắt phải có thể thực hiện lại an toàn.

Cơ chế thay trang

Khi RAM đầy, chọn trang cũ để nhường khung.

Hệ điều hành quản lý trạng thái trang; MMU kiểm tra và phát hiện trang chưa hiện diện.



Ưu điểm và hạn chế của phân trang theo yêu cầu

Ưu điểm

- Giảm lượng RAM cần khi bắt đầu.
- Chạy được nhiều tiến trình hơn.
- Chỉ đọc các trang thật sự cần.

Hạn chế

- Lỗi trang làm chương trình dừng tạm thời.
- Cần bảng trang và cơ chế thay trang.
- Hiệu năng giảm mạnh nếu lỗi trang quá nhiều.

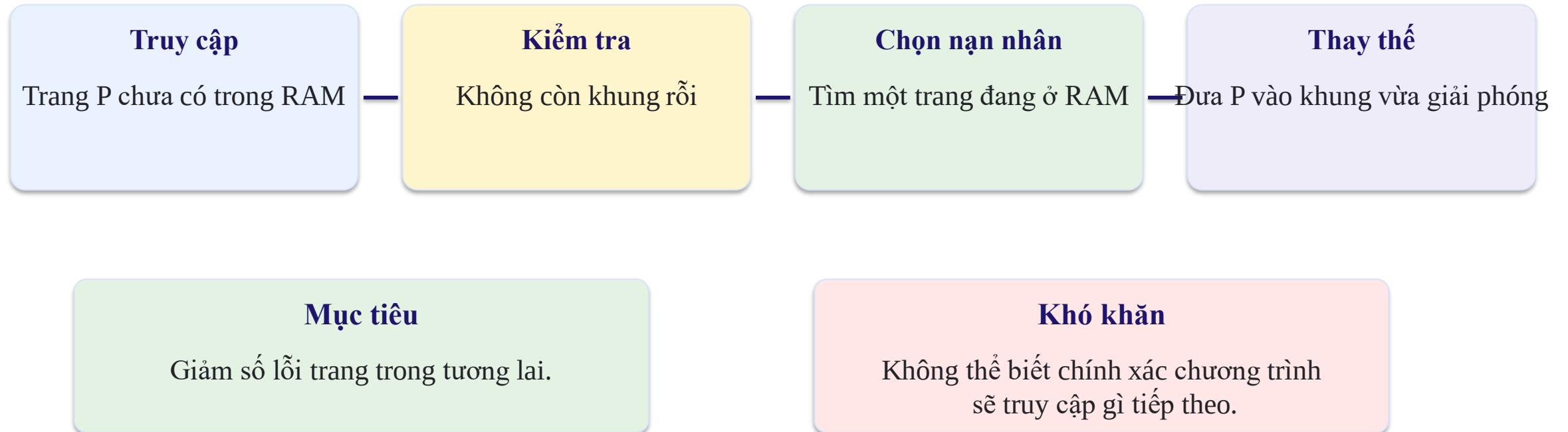
Không phải nạp ít nhất là tốt nhất; cần nạp đủ để vùng đang hoạt động nằm trong RAM.



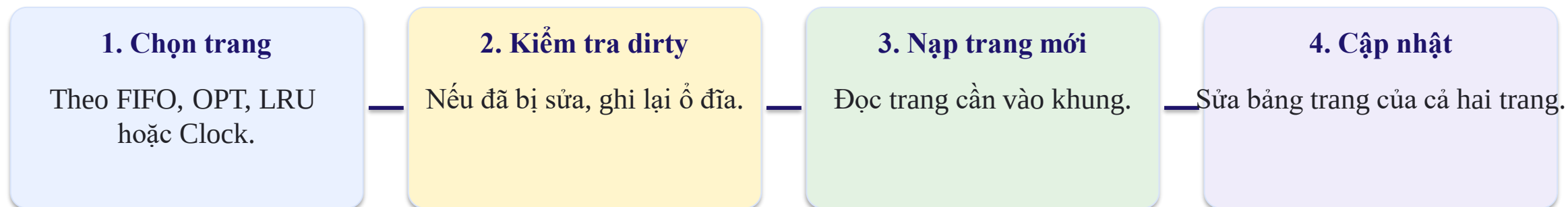
7.3. Thay thế trang

Khi không còn khung rồi, hệ điều hành phải chọn một trang đang ở RAM để đưa ra.

Khi nào cần thay thế trang?



Quy trình thay một trang



Trang “nạn nhân” là trang bị chọn để rời RAM.

Thuật toán tốt là thuật toán chọn trang ít có khả năng được dùng lại sớm.



Chuỗi tham chiếu trang dùng để mô phỏng

Chuỗi trang:

7	0	1	2	0	3	0	4	2	3	0	3	2
---	---	---	---	---	---	---	---	---	---	---	---	---

Cách đọc:

Mỗi số là trang mà CPU cần truy cập tại thời điểm đó.

Nếu trang đã có trong khung → trúng trang.

Nếu chưa có → lỗi trang và phải nạp/thay trang.

Các thuật toán khác nhau ở cách chọn trang nạn nhân khi RAM đầy.



FIFO: Trang vào trước được thay trước

Cách nhớ

Xếp các trang như một hàng đợi.

Khi cần thay

Lấy trang đã nằm trong RAM lâu nhất.

Ưu điểm

Dễ hiểu và dễ cài đặt.

Vào trước

P7

Tiếp theo

P0

Tiếp theo

P1

Thay trước

P7

FIFO không quan tâm trang có đang được dùng thường xuyên hay không.

FIFO – mô phỏng các bước đầu



Trang	7	0	1	2	0	3	0
Khung 1	7	7	7	2	2	2	2
Khung 2		0	0	0	0	3	3
Khung 3			1	1	1	1	0
Kết quả	Lỗi	Lỗi	Lỗi	Lỗi		Lỗi	Lỗi

Sau ba lỗi đầu, ba khung đầy. Lần truy cập trang 2 thay trang 7 vì 7 vào sớm nhất.

FIFO – tiếp tục đến hết chuỗi



Trang	4	2	3	0	3	2
Khung 1	4	4	4	0	0	0
Khung 2	3	2	2	2	2	2
Khung 3	0	0	3	3	3	3
Kết quả	Lỗi	Lỗi	Lỗi	Lỗi		

Hàng đợi luôn ghi nhớ thứ tự các trang được đưa vào RAM.

Kết quả của FIFO



Số khung

3 khung trang

Số lỗi trang

10 lỗi trên 13 lần truy cập

Nhận xét

Đơn giản nhưng đôi khi thay nhầm trang đang cần.

FIFO phù hợp khi:

- Cần cách cài đặt rất đơn giản.
- Không có thông tin về lịch sử truy cập.
- Chấp nhận hiệu quả không tối ưu.



Nghịch lý Belady: Thêm khung nhưng lỗi trang lại tăng

Chuỗi: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

FIFO với 3 khung

9 lỗi trang

FIFO với 4 khung

10 lỗi trang

FIFO có thể gặp trường hợp nhiều khung hơn nhưng kết quả tệ hơn.



OPT: Thay trang sẽ được dùng muộn nhất

Cách chọn

Nhìn về tương lai và thay trang lâu nhất mới được dùng lại.

Kết quả

Cho số lỗi trang ít nhất với cùng chuỗi tham chiếu.

Hạn chế

Thực tế không biết chính xác tương lai.

Vai trò của OPT:

- Dùng làm chuẩn tốt nhất để so sánh.
- Không phải thuật toán triển khai trực tiếp trong hệ điều hành thông thường.

OPT – ví dụ với 3 khung



Trang	7	0	1	2	0	3	0	4	2	3	0	3	2
Khung 1	7	7	7	2	2	2	2	2	2	2	2	2	2
Khung 2		0	0	0	0	0	0	4	4	4	0	0	0
Khung 3			1	1	1	3	3	3	3	3	3	3	3
Kết quả	Lỗi	Lỗi	Lỗi	Lỗi		Lỗi		Lỗi			Lỗi		

OPT tạo 7 lỗi trang – ít nhất trong ba thuật toán được so sánh.



LRU: Thay trang lâu nhất chưa được dùng lại

Cách chọn

Dựa vào quá khứ: trang nào lâu nhất không được truy cập sẽ bị thay.

Ý tưởng

Trang vừa dùng gần đây có khả năng còn được dùng tiếp.

Đánh đổi

Tốt hơn FIFO nhưng phải theo dõi lịch sử truy cập.

Cũ nhất

P7 – dùng từ lâu

Gần đây

P0 – vừa dùng

Mới nhất

P2 – vừa nạp

Nạn nhân

P7

LRU – mô phỏng nửa đầu chuỗi



Trang	7	0	1	2	0	3	0
Khung 1	7	7	7	2	2	2	2
Khung 2		0	0	0	0	0	0
Khung 3			1	1	1	3	3
Kết quả	Lỗi	Lỗi	Lỗi	Lỗi		Lỗi	

Mỗi lần truy cập, trang đó trở thành trang được dùng gần nhất.

LRU – kết quả toàn bộ chuỗi



Trang	7	0	1	2	0	3	0	4	2	3	0	3	2
Khung 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Khung 2		0	0	0	0	0	0	0	0	3	3	3	3
Khung 3			1	1	1	3	3	3	2	2	2	2	2
Kết quả	Lỗi	Lỗi	Lỗi	Lỗi		Lỗi		Lỗi	Lỗi	Lỗi	Lỗi		

LRU tạo 9 lỗi trang: tốt hơn FIFO (10) nhưng kém OPT (7).



Theo dõi LRU bằng cách nào?

Dấu thời gian

Ghi lần truy cập gần nhất của từng trang; thay trang có thời điểm cũ nhất.

Danh sách thứ tự

Đưa trang vừa dùng lên đầu; trang cuối danh sách là nạn nhân.

Xấp xỉ LRU

Dùng bit tham chiếu và thuật toán Clock để giảm chi phí.

LRU chính xác cần cập nhật dữ liệu mỗi lần truy cập nên có thể tốn phần cứng và thời gian.

Trong thực tế, nhiều hệ điều hành dùng cách gần đúng thay vì LRU hoàn toàn chính xác.

So sánh các thuật toán thay thế trang



Thuật toán	Cách chọn	Ưu điểm	Hạn chế
FIFO	Vào RAM lâu nhất	Rất đơn giản	Có thể thay trang đang dùng
OPT	Dùng lại muộn nhất	Ít lỗi nhất	Không biết tương lai
LRU	Lâu nhất chưa dùng	Bám sát tính cục bộ	Theo dõi lịch sử tốn chi phí

Với chuỗi ví dụ và 3 khung: FIFO 10 lỗi, OPT 7 lỗi, LRU 9 lỗi.

Bài tập: Mô phỏng thay thế trang



Chuỗi tham chiếu: 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

Số khung: 3, ban đầu đều rỗng.

Thực hiện:

1. FIFO và đếm số lỗi trang.
2. LRU và đếm số lỗi trang.
3. Chỉ ra lần đầu hai thuật toán chọn nạn nhân khác nhau.
4. Giải thích thuật toán nào phù hợp hơn với tính cục bộ.



7.4. Cấp phát khung

Hệ điều hành quyết định mỗi tiến trình được giữ bao nhiêu khung trong RAM.



Vì sao phải phân chia khung?

RAM có hạn

Tổng số khung nhỏ hơn tổng số Trang của các tiến trình.

Mỗi tiến trình cần khác nhau

Tiến trình lớn hoặc đang hoạt động mạnh thường cần nhiều khung hơn.

Phân chia ảnh hưởng hiệu năng

Quá ít khung → nhiều lỗi trang;
quá nhiều → lãng phí.

Cấp phát khung là bài toán cân bằng giữa công bằng và hiệu quả.

Cấp phát bằng nhau



Ví dụ: 12 khung cho 3 tiến trình

P1

Nhận 4 khung

P2

Nhận 4 khung

P3

Nhận 4 khung

Ưu điểm: Dễ thực hiện và nhìn có vẻ công bằng.

Hạn chế: Tiến trình lớn và nhỏ đều nhận như nhau, dù nhu cầu khác nhau.

Cấp phát theo kích thước tiến trình



Tổng 12 khung – kích thước: P1 = 10 trang, P2 = 20 trang, P3 = 30 trang

P1

$$10/60 \times 12 = 2 \text{ khung}$$

P2

$$20/60 \times 12 = 4 \text{ khung}$$

P3

$$30/60 \times 12 = 6 \text{ khung}$$

Cấp phát theo tỷ lệ phản ánh kích thước, nhưng chưa phản ánh mức hoạt động tại từng thời điểm.



Mỗi tiến trình cần một số khung tối thiểu

Vì sao?

Một lệnh có thể cần đồng thời trang chứa lệnh và các trang chứa toán hạng.

Nếu quá ít khung

Tiến trình có thể liên tục nạp rồi thay chính trang vừa cần.

Phụ thuộc kiến trúc

Số khung tối thiểu phụ thuộc dạng lệnh và số lần truy cập bộ nhớ.

Ví dụ đơn giản: Một lệnh nằm ở hai trang và đọc dữ liệu ở hai trang khác → có thể cần ít nhất 4 khung.

Thay thế cục bộ và thay thế toàn cục



Thay thế cục bộ

Tiến trình chỉ được thay trang trong các khung của chính nó.

Số khung của tiến trình ổn định hơn.

Thay thế toàn cục

Tiến trình có thể lấy khung từ tiến trình khác.

Linh hoạt hơn nhưng các tiến trình ảnh hưởng lẫn nhau.

Toàn cục thường cho thông lượng tốt hơn; cục bộ dễ dự đoán hơn.

So sánh thay thế cục bộ và toàn cục



Tiêu chí	Cục bộ	Toàn cục
Nạn nhân	Trang của chính tiến trình	Trang của bất kỳ tiến trình nào
Số khung	Tương đối ổn định	Thay đổi theo tải
Ảnh hưởng lẫn nhau	Ít	Nhiều
Khả năng thích nghi	Hạn chế	Tốt hơn
Dễ dự đoán	Cao hơn	Thấp hơn

Không có lựa chọn luôn tốt nhất; quyết định phụ thuộc mục tiêu của hệ thống.



Tập làm việc: Những trang tiến trình đang cần

Cửa sổ quan sát

Xét một số lần truy cập gần nhất, ví dụ 10.000 lần.

Tập làm việc

Tập các trang xuất hiện trong cửa sổ đó.

Ý nghĩa

Nên cấp đủ khung để giữ tập trang đang hoạt động.

Chuỗi gần đây: 2, 3, 2, 4, 3, 2 $\rightarrow$ tập làm việc = {2, 3, 4}



Tập làm việc thay đổi theo giai đoạn chương trình

Giai đoạn 1: Khởi tạo

0 1 2 1 0 2

WS = {0,1,2}

Giai đoạn 2: Xử lý dữ liệu

5 6 7 6 5 8

WS = {5,6,7,8}

Giai đoạn 3: Xuất kết quả

9 10 9 11 10 9

WS = {9,10,11}

Tiến trình không cần cùng một số khung cố định trong suốt thời gian chạy.



Thrashing: Máy bận thay trang nhưng công việc chạy rất ít

Biểu hiện

Lỗi trang liên tục và ổ đĩa hoạt động mạnh.

Nguyên nhân chính

Tổng tập làm việc của các tiến trình lớn hơn số khung RAM.

Hệ quả

CPU nhàn rỗi vì phải chờ I/O; hệ thống phản hồi rất chậm.

Nạp trang → chạy rất ít → thiếu trang khác → thay trang → lặp lại

Dấu hiệu nhận biết thrashing



Tỷ lệ lỗi trang tăng

Nhiều truy cập phải chờ nạp trang.

Ổ đĩa bận cao

Đọc/ghi swap liên tục.

CPU sử dụng thấp

CPU chờ I/O thay vì thực thi.

Phản hồi chậm

Ứng dụng treo hoặc giạt kéo dài.

Tăng thêm tiến trình trong lúc này thường làm tình hình xấu hơn.

Ít RAM cho mỗi tiến trình → nhiều lỗi trang → CPU chờ → hệ điều hành lại nạp thêm tiến trình → càng nhiều lỗi trang



Những nguyên nhân thường gây thrashing

Quá nhiều tiến trình

Mỗi tiến trình chỉ còn rất ít khung.

Tập làm việc lớn

Ứng dụng đang xử lý vùng dữ liệu rộng.

Cấp phát chưa phù hợp

Khung được chia không theo nhu cầu thực tế.

Thuật toán thay trang kém

Liên tục loại bỏ trang sắp được dùng lại.

Thiếu kiểm soát tải

Vẫn nhận thêm tiến trình dù RAM đã quá tải.

Cách hạn chế thrashing



Giảm số tiến trình

Tạm dừng hoặc swap out một số tiến trình.

Cấp thêm khung

Ưu tiên tiến trình có lỗi trang cao.

Theo dõi tập làm việc

Giữ các trang đang hoạt động trong RAM.

Theo dõi tần suất lỗi

Điều chỉnh khung khi lỗi quá cao hoặc quá thấp.

Mục tiêu không phải loại bỏ mọi lỗi trang, mà giữ chúng ở mức hợp lý.

PFF: Điều chỉnh khung theo tần suất lỗi trang



Lỗi trang quá cao

Cấp thêm khung cho tiến trình nếu còn khung rỗi.

Lỗi trang trong khoảng tốt

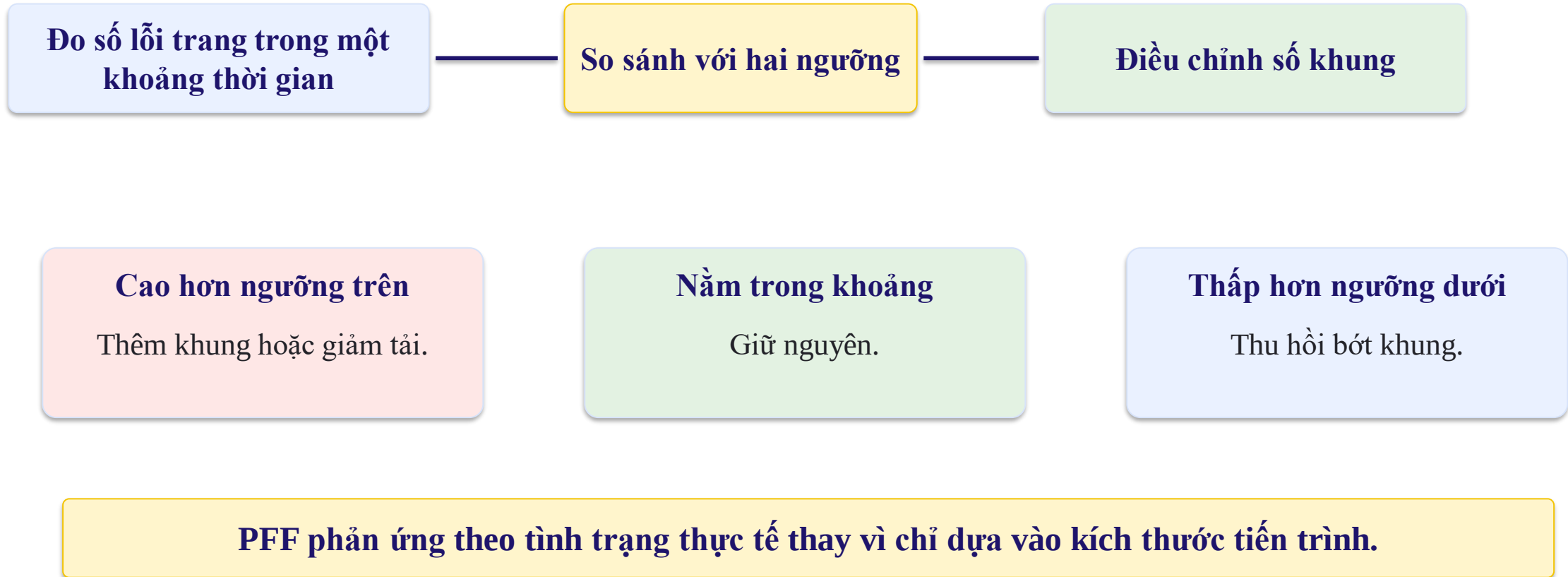
Giữ nguyên số khung.

Lỗi trang quá thấp

Có thể thu hồi bớt khung để cấp cho tiến trình khác.

Ngưỡng thấp ← vùng hoạt động tốt → ngưỡng cao

Quy trình điều khiển theo tần suất lỗi trang



Ví dụ điều chỉnh khung giữa ba tiến trình



Tiến trình	Khung hiện tại	Lỗi trang / giây	Quyết định
P1	6	2	Có thể giảm 1 khung
P2	4	18	Cần thêm khung
P3	5	7	Giữ nguyên

Có thể chuyển một khung từ P1 sang P2 để giảm lỗi trang tổng thể.



7.5. Phân đoạn theo yêu cầu

Mỗi đoạn logic của chương trình chỉ được nạp khi chương trình thực sự sử dụng đoạn đó.



Phân đoạn theo yêu cầu là gì?

Chia theo ý nghĩa

Mã, dữ liệu, heap, stack hoặc từng mô-đun là các đoạn riêng.

Nạp khi cần

Đoạn chưa dùng có thể nằm trên ổ đĩa.

Khi truy cập

Nếu đoạn chưa ở RAM, hệ điều hành nạp đoạn đó.

Khác phân trang, các đoạn có kích thước khác nhau và gắn với cấu trúc logic của chương trình.

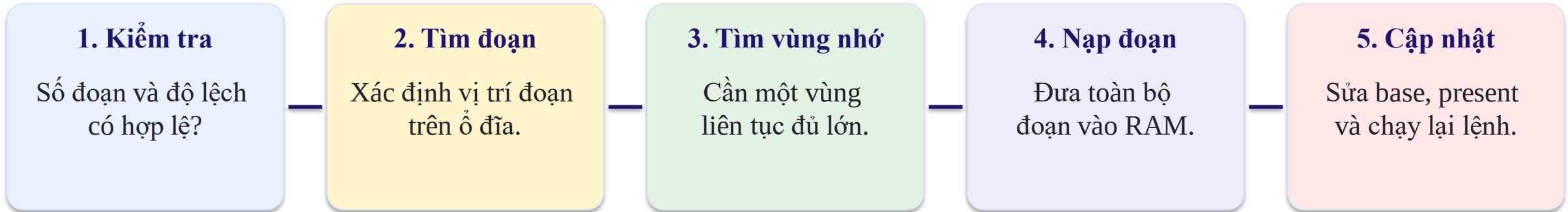
Bảng đoạn cần lưu những gì?



Đoạn	Base	Limit	Có trong RAM?	Quyền
0 – Mã	12000	8000	Có	R-X
1 – Dữ liệu	—	5000	Chưa	RW-
2 – Heap	25000	6000	Có	RW-
3 – Stack	—	4000	Chưa	RW-

Nếu đoạn chưa hiện diện, base chưa có giá trị vật lý và truy cập sẽ tạo lỗi đoạn.

Các bước xử lý lỗi đoạn

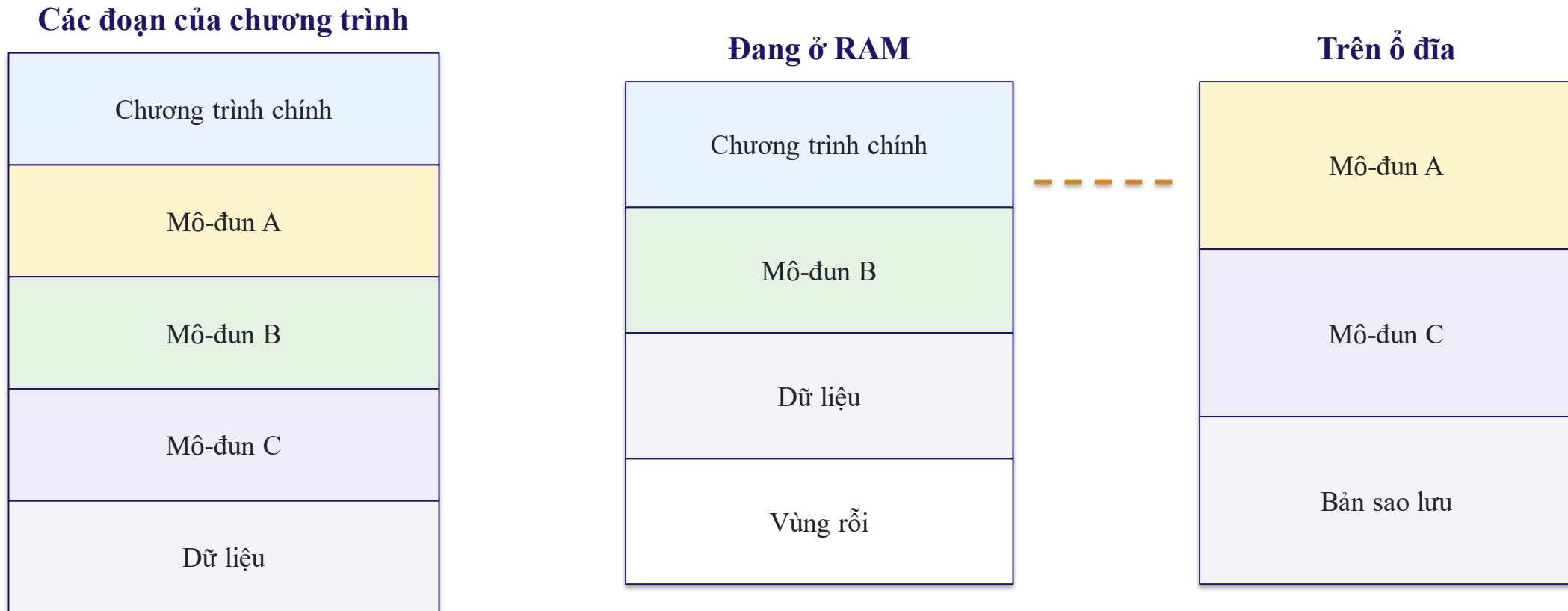


Vì đoạn có kích thước biến đổi, việc tìm chỗ trống khó hơn phân trang.

Nếu không có vùng liên tục đủ lớn, hệ điều hành có thể kết khối hoặc kết hợp phân đoạn với phân trang.



Ví dụ: Chỉ nạp mô-đun đang được gọi



Khi chương trình gọi mô-đun A, hệ điều hành nạp toàn bộ đoạn A vào một vùng RAM liên tục.

Phân trang theo yêu cầu và phân đoạn theo yêu cầu

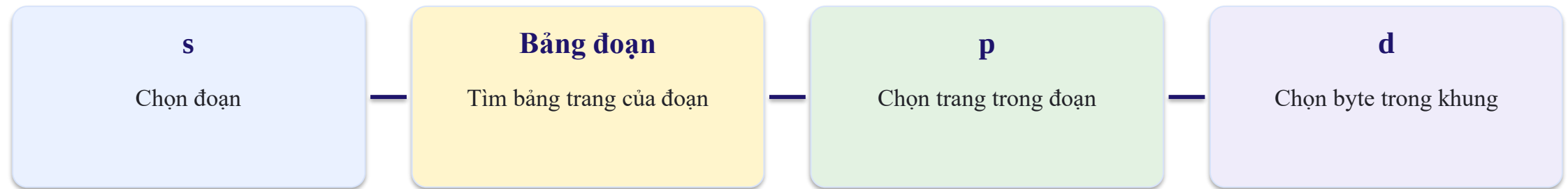


Tiêu chí	Phân trang theo yêu cầu	Phân đoạn theo yêu cầu
Đơn vị	Trang cố định	Đoạn có kích thước khác nhau
Ý nghĩa	Trong suốt với lập trình viên	Gắn với mã, dữ liệu, mô-đun
Nơi đặt trong RAM	Bất kỳ khung rồi	Cần vùng liên tục đủ lớn
Phân mảnh	Chủ yếu phân mảnh trong	Phân mảnh ngoài
Lỗi khi thiếu	Lỗi trang	Lỗi đoạn

Kết hợp phân đoạn và phân trang



Địa chỉ logic: s | p | d



Lợi ích:

- Giữ được cách tổ chức theo đoạn.
- Không cần đặt toàn bộ đoạn trong một vùng vật lý liên tục.
- Có thể nạp từng trang của đoạn khi cần.



Ưu điểm và hạn chế của phân đoạn theo yêu cầu

Ưu điểm

- Nạp đúng mô-đun hoặc vùng dữ liệu cần dùng.
- Quyền bảo vệ và chia sẻ theo ý nghĩa logic.
- Phù hợp chương trình có nhiều mô-đun độc lập.

Hạn chế

- Mỗi đoạn có kích thước khác nhau.
- Cần vùng RAM liên tục đủ lớn.
- Có phân mảnh ngoài và có thể phải kết khối.

Hệ thống hiện đại thường ưu tiên phân trang, hoặc kết hợp phân đoạn với phân trang.

Tổng kết Chương 7



Chỉ giữ phần đang dùng trong RAM; phần còn lại nằm trên ổ đĩa.

Trang chưa có gây lỗi trang; hệ điều hành nạp trang rồi chạy lại lệnh
FIFO đơn giản; OPT là chuẩn tốt nhất;

LRU bám sát lịch sử sử dụng.

Phải cân bằng số khung, tập làm việc và tần suất lỗi trang.

Xảy ra khi các tiến trình không có đủ khung cho vùng đang hoạt động.

Nạp từng đoạn logic khi cần; linh hoạt nhưng khó tìm vùng liên tục



Câu hỏi và bài tập thảo luận

Vì sao chỉ một tỷ lệ lỗi trang rất nhỏ cũng làm hệ thống chậm rõ rệt?

Mô phỏng FIFO, OPT và LRU với chuỗi: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5; dùng 3 khung.

Dấu hiệu nào cho thấy hệ thống đang thrashing? Nêu hai cách xử lý.

Phân biệt lỗi trang và lỗi đoạn.

Vì sao kết hợp phân đoạn với phân trang giúp giảm phân mảnh ngoài?

Email: minhnl@utc.edu.vn